COMPARATIVE PERFORMANCE ANALYSIS OF THREE SORTING ALGORITHMS

Assignment : 01

Bubble Sort, Selection Sort and Insertion Sort

Abdur Rahman — 22052020002

1. Introduction

Sorting is a common operation in computer science. This experiment compared Bubble Sort, Selection Sort and Insertion Sort by measuring their execution time for different input sizes and data arrangements. The tests used 5,000, 25,000, 50,000 and 75,000 elements with unsorted, sorted and partially sorted data.

2. Important Definitions

2.1 Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them when they are in the wrong order. With the swapped flag used in this program, it can stop early when the array is already sorted.

2.2 Selection Sort

Selection Sort divides the array into sorted and unsorted parts. It finds the smallest element from the unsorted part and places it in the next sorted position.

2.3 Insertion Sort

Insertion Sort builds the sorted part one element at a time. Each new element is inserted into its correct position by shifting larger elements to the right.

2.4 Time Complexity

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort (with swapped flag)	O(n)	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$

4. Results

4.1 Unsorted (Random) Data

Input Size	Bubble Sort (ms)	Selection Sort (ms)	Insertion Sort (ms)
5000	102.06	49.06	38.78
25000	4575.2	1479.92	1030.41
50000	15231.5	5620.26	4108.72
75000	35098.8	16034.4	10224.4

4.2 Sorted Data

Input Size	Bubble Sort (ms)	Selection Sort (ms)	Insertion Sort (ms)
5000	0.02	49.85	0.0444
25000	0.0988	1579.09	0.2569
50000	0.1987	6962.46	0.4146
75000	0.5137	17694.7	0.4947

4.3 Partially Sorted Data

Input Size	Bubble Sort (ms)	Selection Sort (ms)	Insertion Sort (ms)
5000	89.19	52.28	4.2
25000	1910.24	1610.48	135.06
50000	7255.5	5789.28	514.43
75000	19729.9	14639.8	1263.38

5. Graph Comparison

The following graphs were generated directly from the measured values in the original assignment.



Figure 1: Execution time for unsorted (random) data.

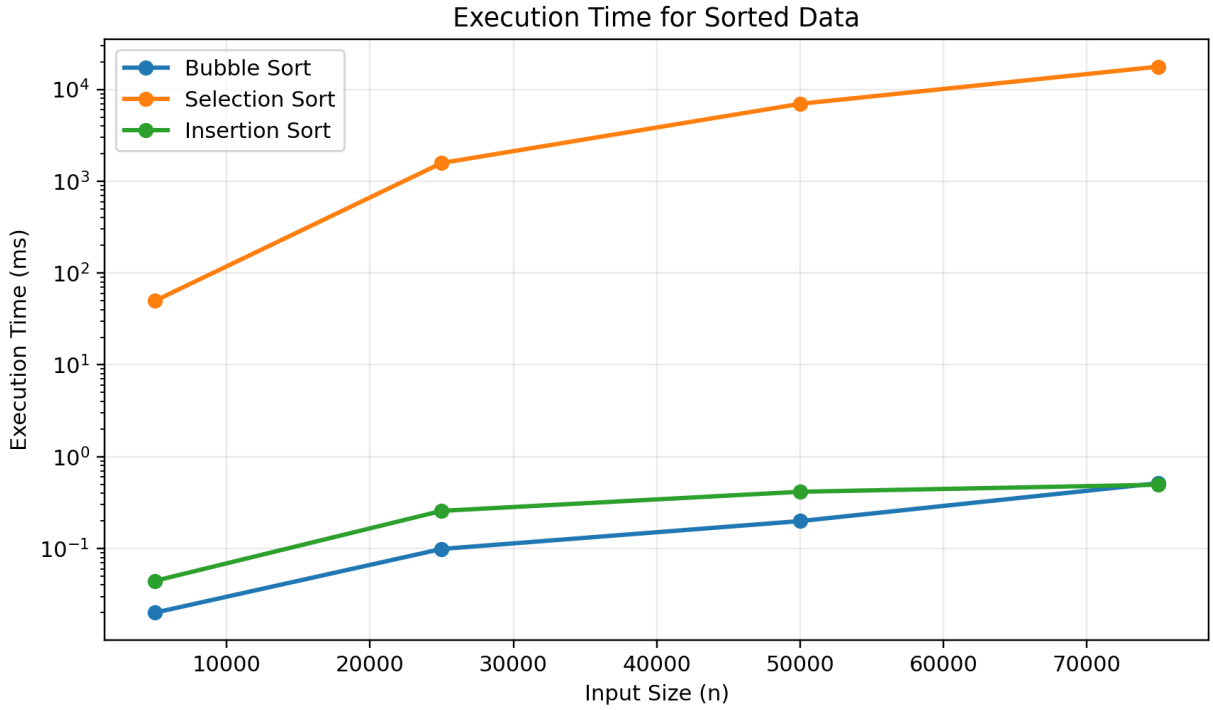


Figure 2: Execution time for sorted data. A logarithmic y-axis is used because Bubble Sort and Insertion Sort are below 1 ms while Selection Sort reaches several seconds.

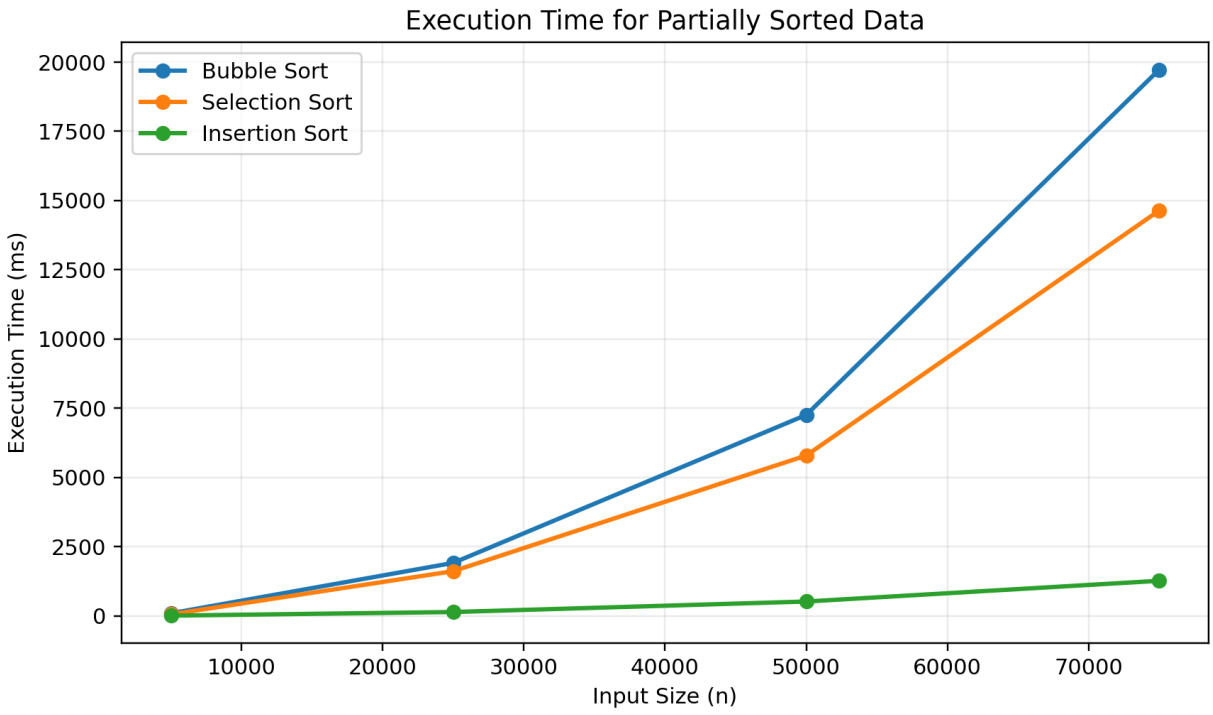


Figure 3: Execution time for partially sorted data.

6. Comparison

Data Condition	Fastest	Slowest	Main Observation
Unsorted	Insertion Sort	Bubble Sort	All increased rapidly as input size increased.
Sorted	Bubble Sort / Insertion Sort	Selection Sort	Bubble and Insertion used their O(n) best-case behavior.
Partially Sorted	Insertion Sort	Bubble Sort	Insertion Sort required much less movement of elements.

For unsorted data at 75,000 elements, Bubble Sort took about 35.10 s, Selection Sort 16.03 s and Insertion Sort 10.22 s. For sorted data, Bubble Sort and Insertion Sort remained below 1 ms, while Selection Sort took about 17.69 s. For partially sorted data, Insertion Sort took about 1.26 s compared with 14.64 s for Selection Sort and 19.73 s for Bubble Sort.

7. Source Code C++

```
#include <iostream>
#include <vector>
#include <chrono>
#include <cstdlib>
using namespace std;

// Sorting Algorithms
// Bubble Sort with swapped flag
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

// Selection Sort
void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
```

```
            }
        }
        swap(arr[i], arr[minIdx]);
    }
}

// Insertion Sort
void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// Helper functions to generate array data
// Generates random unsorted data
vector<int> generateUnsorted(int size) {
    vector<int> arr(size);
    for (int i = 0; i < size; i++) {
        arr[i] = rand() % 1000000 + 1;
    }
    return arr;
}

// Generates already sorted data
vector<int> generateSorted(int size) {
    vector<int> arr(size);
    for (int i = 0; i < size; i++) {
        arr[i] = i;
    }
    return arr;
}

// Generates partially sorted data (swaps 5% of elements)
vector<int> generatePartiallySorted(int size) {
    vector<int> arr = generateSorted(size);
    int swaps = size * 0.05;
    for (int i = 0; i < swaps; i++) {
        int idx1 = rand() % size;
        int idx2 = rand() % size;
        swap(arr[idx1], arr[idx2]);
    }
    return arr;
}
```

```
// Helper function to measure execution time
double measureTimeInMs(void (*sortFunc)(vector<int>&), vector<int> arr) {
    auto start = chrono::high_resolution_clock::now();
    sortFunc(arr); // pass array copy so original is not changed
    auto end = chrono::high_resolution_clock::now();
    chrono::duration<double, milli> duration = end - start;
    return duration.count();
}

void runBenchmark(string conditionName, vector<int> data) {
    cout << "\nCondition: " << conditionName << " (Size: " << data.size() << ")\n";
    cout << "-----------------------------------------------\n";
    double timeBubble = measureTimeInMs(bubbleSort, data);
    double timeSelection = measureTimeInMs(selectionSort, data);
    double timeInsertion = measureTimeInMs(insertionSort, data);
    cout << "Bubble Sort: " << timeBubble << " ms\n";
    cout << "Selection Sort: " << timeSelection << " ms\n";
    cout << "Insertion Sort: " << timeInsertion << " ms\n";
}

// Main function
int main() {
    srand(42); // Seed random number generator
    int sizes[] = {5000,25000,50000,75000};
    cout << "SORTING ALGORITHM PERFORMANCE TEST \n";
    cout << "Abdur Rahman---22052020002\n";
    for (int size : sizes) {
        cout << "\n TESTING FOR N = " << size;
        cout << "\n----------------------------------------";
        vector<int> unsortedData = generateUnsorted(size);
        runBenchmark("Unsorted (Random)", unsortedData);
        vector<int> sortedData = generateSorted(size);
        runBenchmark("Sorted", sortedData);
        vector<int> partiallySortedData = generatePartiallySorted(size);
        runBenchmark("Partially Sorted", partiallySortedData);
    }
    return 0;
}
```

8. Output

```
SORTING ALGORITHM PERFORMANCE TEST
Abdur Rahman---22052020002
TESTING FOR N = 5000
----------------------------------------
Condition: Unsorted (Random) (Size: 5000)
----------------------------------------------
Bubble Sort: 102.057 ms
Selection Sort: 49.0555 ms
Insertion Sort: 38.7805 ms
Condition: Sorted (Size: 5000)
----------------------------------------------
Bubble Sort: 0.019951 ms
Selection Sort: 49.8459 ms
Insertion Sort: 0.04441 ms
Condition: Partially Sorted (Size: 5000)
----------------------------------------------
Bubble Sort: 89.1861 ms
Selection Sort: 52.2792 ms
Insertion Sort: 4.19888 ms
TESTING FOR N = 25000
----------------------------------------
Condition: Unsorted (Random) (Size: 25000)
----------------------------------------------
Bubble Sort: 4575.2 ms
Selection Sort: 1479.92 ms
Insertion Sort: 1030.41 ms
Condition: Sorted (Size: 25000)
----------------------------------------------
Bubble Sort: 0.09876 ms
Selection Sort: 1579.09 ms
Insertion Sort: 0.25688 ms
Condition: Partially Sorted (Size: 25000)
----------------------------------------------
Bubble Sort: 1910.24 ms
Selection Sort: 1610.48 ms
Insertion Sort: 135.058 ms
TESTING FOR N = 50000
----------------------------------------
Condition: Unsorted (Random) (Size: 50000)
----------------------------------------------
Bubble Sort: 15231.5 ms
Selection Sort: 5620.26 ms
Insertion Sort: 4108.72 ms
Condition: Sorted (Size: 50000)
----------------------------------------------
Bubble Sort: 0.198749 ms
Selection Sort: 6962.46 ms
Insertion Sort: 0.41462 ms
Condition: Partially Sorted (Size: 50000)
----------------------------------------------
Bubble Sort: 7255.5 ms
Selection Sort: 5789.28 ms
```

```
Insertion Sort: 514.429 ms
TESTING FOR N = 75000
-----------------------------------------
Condition: Unsorted (Random) (Size: 75000)
-----------------------------------------------
Bubble Sort: 35098.8 ms
Selection Sort: 16034.4 ms
Insertion Sort: 10224.4 ms
Condition: Sorted (Size: 75000)
-----------------------------------------------
Bubble Sort: 0.51366 ms
Selection Sort: 17694.7 ms
Insertion Sort: 0.49469 ms
Condition: Partially Sorted (Size: 75000)
-----------------------------------------------
Bubble Sort: 19729.9 ms
Selection Sort: 14639.8 ms
Insertion Sort: 1263.38 ms
```

9. Conclusion

The experiment showed that sorting performance depended on both input size and data arrangement. Insertion Sort performed fastest on unsorted and partially sorted data, while Bubble Sort and Insertion Sort showed very small times for already sorted data. Selection Sort remained $O(n^2)$ for all cases and became much slower as the input size increased. The measured results followed the expected time-complexity trends, although exact execution times depended on the computer and measurement conditions.

10. Limitation

Each case was run only once, so the measured times contained some measurement noise. Absolute execution times could also change with hardware, compiler and optimization settings. The trends were the main focus of the experiment.